NAModule4NetworkXTutorial

October 16, 2026

[173]:
```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from networkx.drawing.nx_pydot import graphviz_layout
import ndlib.models.ModelConfig as mc
import ndlib.models.epidemics as ep
import operator
import random
```

1 Module 4 NetworkX Tutorial

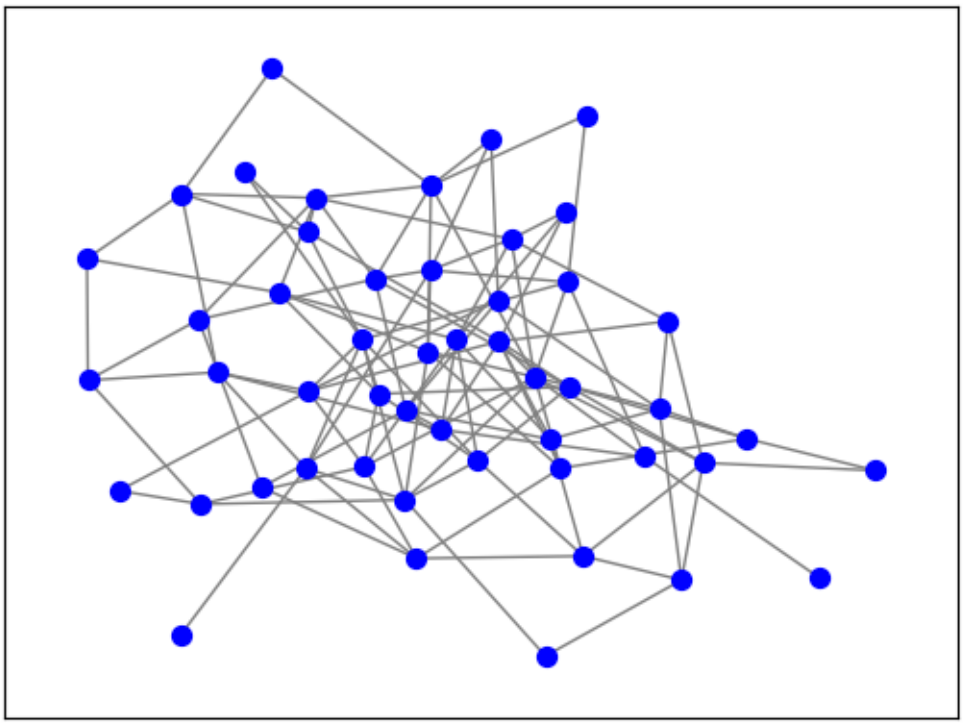
In this tutorial, we will go through examples that may assist you in completing this module's assignment.

2 General Assignment Tutorial: Configuring a model

Configuring the models is a task you're going to need to complete for each question in this week's assignment. This section will guide you on how to accomplish this.

First, let's create a dummy graph for demonstration and visualize its structure.

[174]:
```
G = nx.dense_gnm_random_graph(50, 120, seed=42)
pos = nx.spring_layout(G)
nx.draw_networkx(G, pos=pos, with_labels=False, node_color='blue',
                 edge_color='gray', node_size=50)
```



Now, let's configure the SI model, a very simple model where nodes infect others in each iteration with a set probability. Here, that probability will be called 'beta,' a parameter we can add before selecting seed nodes. For this assignment, all parameters for the 'config' object will be added before selecting seed nodes.

To start a diffusion, we need to have a set of seed nodes. There are two ways to add seed nodes to our configuration. We can either specify the seed nodes or define the fraction of nodes to be set as seed nodes.

To select specific seed nodes, we use the method

```
config = mc.Configuration()
config.add_model_initial_configuration("Infected", seed_node_set)
```

To define a range of infected nodes, we use the method

```
config = mc.Configuration()
config.add_model_parameter("fraction_infected", ratio_of_infection)
```

Now, let's configure our SI model:

[175]:
```
# Selecting the model
model = ep.SIModel(G, seed=42)

# Configuring the model

# Adding Parameters
```

```
config = mc.Configuration()
config.add_model_parameter('beta', 0.5)

# Selecting Seed Nodes
seed_nodes = [0,1,2,3,4]
config.add_model_initial_configuration("Infected", seed_nodes)

# To use the other model of seed node selection, you would commentize the two␣
 ↪lines of
#  code above and uncomment the below line.
# config.add_model_parameter("fraction_infected", 0.1)


model.set_initial_status(config)
```

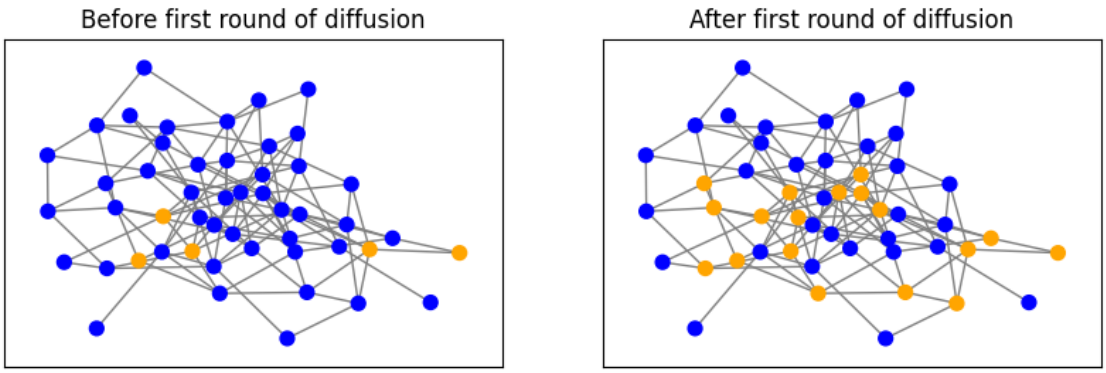
Now, let's examine some properties of the simulation. By running the following code, we simulate the first 10 iterations:

[176]:
```
iterations = model.iteration_bunch(10)
```

Here, iterations will be a list of dictionaries representing the first 10 iterations of the model spreading. Below, we can visualize the infection of the nodes using the first two iterations. Understanding the code below is not necessary, but the visuals themselves should help you realize how the code successfully simulates diffusion models.

[177]:
```
color_list = ['blue', 'orange']
fig, axes = plt.subplots(1, 2, figsize=[10,3])
node_list_before = [color_list[v] for _, v in iterations[0]['status'].items()]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[0])
axes[0].set_title("Before first round of diffusion")

for k, v in iterations[1]['status'].items():
    node_list_before[k] = color_list[v]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[1])
axes[1].set_title("After first round of diffusion");
```



Now, we can delve a bit deeper into what each dictionary in iteration is. Let's take a look at the first dictionary, which corresponds to the first iteration:

[178]:
```
it1 = iterations[1]

print(it1)
```

```
{'iteration': 1, 'status': {10: 1, 13: 1, 18: 1, 21: 1, 26: 1, 31: 1, 32: 1, 36:
1, 39: 1, 41: 1, 42: 1, 44: 1, 47: 1}, 'node_count': {0: 32, 1: 18},
'status_delta': {0: -13, 1: 13}}
```

As you can see, each iteration is a dictionary storing information about the iteration. Remember that a python dictionary is a collection of values referenced by a key. Let's break each field down.

[179]:
```
print((it1['iteration']))
```

```
1
```

The 'iteration' item is self explanatory, representing which iteration has just been passed.

[180]:
```
print(it1['status'])

print("Status of node 10: " + str(it1['status'][10]))
```

```
{10: 1, 13: 1, 18: 1, 21: 1, 26: 1, 31: 1, 32: 1, 36: 1, 39: 1, 41: 1, 42: 1,
44: 1, 47: 1}
Status of node 10: 1
```

The 'status' field is a dictionary containing the nodes that have had their status changed in the previous iteration as keys. The value is either 1 or 0, indicating the node's infection status. Let's take a look at iteration 1's field:

We can see that `10:1` is in the dictionary. This means that node 10 was infected in the last iteration. In some models, we might see `10:0` in 'status.' This would mean that node 10 had lost its infection in the last iteration. Of course, since we are currently using the SI model, we do not see any nodes going back to status 0. There are many more nodes in the network than are shown here, but they aren't in the dictionary because their status did not change in the last iteration.

[181]:
```
print(it1['node_count'])

print("Number of infected nodes: " + str(it1['node_count'][1]))
print("Number of non-infected nodes: " + str(it1['node_count'][0]))
```

```
{0: 32, 1: 18}
Number of infected nodes: 18
Number of non-infected nodes: 32
```

The 'node_count' field is a dictionary representing the number of infected and non-infected nodes. The entry for 0 represents the number of non-infected nodes, and the entry for 1 represents the number of infected nodes. As you can see above, the value for `it1['node_count'][1]` is 18, meaning that 18 nodes are infected.

[182]:
```
print(it1['status_delta'])

print("Change in no. of uninfected nodes: " + str(it1['status_delta'][0]))
```

```
{0: -13, 1: 13}
Change in no. of uninfected nodes: -13
```

'status_delta' is similar to 'node_count,' except the values for the entries of 1 and 0 represent the change from the previous iteration to the one just completed. In the example above, we can see that 13 nodes became infected, so the change in the number of non-infected nodes is -13, which is the value for the item with the key of 0.

3 Task 1 Tutorial: Threshold Model

The threshold model requires you to add a "threshold" parameter to each node. Below, we will demonstrate how to accomplish this.

First, let's select and configure our model:

[183]:
```
del model, iterations

model = ep.ThresholdModel(G, seed = 42)

config = mc.Configuration()
seed_nodes = list(G.nodes)[:5]
config.add_model_initial_configuration("Infected", seed_nodes)
```

Now, we can loop through "G.nodes" and add a threshhold for each node like so:

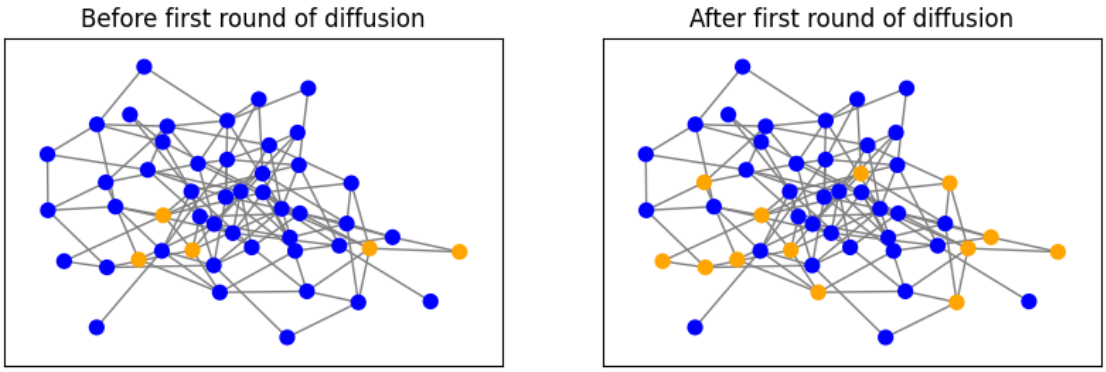
[184]:
```
threshold = 0.25
for node in G.nodes:
    config.add_node_configuration("threshold", node, threshold)

model.set_initial_status(config)
```

[185]:
```
iterations = model.iteration_bunch(10)

color_list = ['blue', 'orange']
fig, axes = plt.subplots(1, 2, figsize=[10,3])
node_list_before = [color_list[v] for _, v in iterations[0]['status'].items()]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[0])
axes[0].set_title("Before first round of diffusion")

for k, v in iterations[1]['status'].items():
    node_list_before[k] = color_list[v]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[1])
axes[1].set_title("After first round of diffusion");
```



4 Task 2 Tutorial: Independent Cascade Model

Simulating the IC model requires you to add a "threshold" parameter to each edge. Below, we will demonstrate how to accomplish this.

First, let's select and configure our model:

[231]:
```
del model, iterations

model = ep.IndependentCascadesModel(G, seed = 42)

config = mc.Configuration()
seed_nodes = list(G.nodes)[:5]
config.add_model_initial_configuration("Infected", seed_nodes)
```

Now, we can loop through each edge and attach each edge's threshold.

[232]:
```
threshold = 0.25
for edge in G.edges():
    config.add_edge_configuration("threshold", edge, threshold)


model.set_initial_status(config)
```

[ ]:
```
iterations = model.iteration_bunch(10)

color_list = ['blue', 'orange', 'orange']
fig, axes = plt.subplots(1, 2, figsize=[10,3])
node_list_before = [color_list[v] for _, v in iterations[0]['status'].items()]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[0])
axes[0].set_title("Before first round of diffusion")

for k, v in iterations[1]['status'].items():
    node_list_before[k] = color_list[v]
nx.draw_networkx(G, pos=pos, with_labels=False, node_color=node_list_before,
                 edge_color='gray', node_size=50, ax=axes[1])
axes[1].set_title("After first round of diffusion");
```


```
{'iteration': 0, 'status': {0: 1, 1: 1, 2: 1, 3: 1, 4: 1, 5: 0, 6: 0, 7: 0, 8:
0, 9: 0, 10: 0, 11: 0, 12: 0, 13: 0, 14: 0, 15: 0, 16: 0, 17: 0, 18: 0, 19: 0,
20: 0, 21: 0, 22: 0, 23: 0, 24: 0, 25: 0, 26: 0, 27: 0, 28: 0, 29: 0, 30: 0, 31:
0, 32: 0, 33: 0, 34: 0, 35: 0, 36: 0, 37: 0, 38: 0, 39: 0, 40: 0, 41: 0, 42: 0,
43: 0, 44: 0, 45: 0, 46: 0, 47: 0, 48: 0, 49: 0}, 'node_count': {0: 45, 1: 5, 2:
0}, 'status_delta': {0: 0, 1: 0, 2: 0}}
{'iteration': 1, 'status': {0: 2, 1: 2, 2: 2, 3: 2, 4: 2, 22: 1, 27: 1, 28: 1,
32: 1, 37: 1, 40: 1, 42: 1, 47: 1}, 'node_count': {0: 37, 1: 8, 2: 5},
'status_delta': {0: -8, 1: 3, 2: 5}}
```